

Progress Report 2

Automated Product Inquiry Response System for Light Recycling Program



Student Name: Namesh Mathara Arachchi Vidanalage

Student ID: 300359798

Course: CSIS 4495 – Applied Research Project

Section: 002

Contents

Work Logs – Summarized.....	3
Summary	3
1. Demo & Feedback.....	3
2. Research & Learning	3
3. Model Implementation & Evaluation.....	4
4. Web Portal Enhancement	4
5. Reporting & Documentation	4
Work Logs – Detailed	5
Research & Learning	5
Word and Sentence Embedding Models in NLP.....	5
Why SBERT (Sentence-BERT)?	6
What is Hugging Face?	6
Why Use Hugging Face for SBERT?	6
SBERT Models available on SentenceTransformer Library in Hugging Face.....	7
Model Implementation & Evaluation.....	7
Evaluation Metric – Mean Reciprocal Rank (MRR)	8
Evaluation Metric – Top-K Accuracy.....	8
Evaluating 9 SBERT Models using Sample Products	9
Final Model Selection for Text Similarity	12
Repository Check-in Details.....	13
References	13

Work Logs – Summarized

Phase	Date	#	Number of Hours	Tr	Description of Work Done
Progress Report 2	25/02/2025	1			Demo for my manager at PCA to get his feedback
Progress Report 2	26/02/2025	2			Researched on computing similarity of texts using NLP - Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities
Progress Report 2	27/02/2025	2			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - all-MiniLM-L6-v2
Progress Report 2	28/02/2025	1			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - all-MiniLM-L6-v2
Progress Report 2	01/03/2025	1			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - BERT-Based Models
Progress Report 2	02/03/2025	1			Modified the ml_model script to consider all textual fields (product_name and product_description)
Progress Report 2	03/03/2025	2			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - BERT-Based Models
Progress Report 2	04/03/2025	1			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - MPNet-Based Models
Progress Report 2	05/03/2025	2			Read about Hugging Face's Transformers that provide sentence embeddings and text similarity capabilities - MPNet-Based Models
Progress Report 2	06/03/2025	2			Started the course - Natural Language Processing: NLP With Transformers in Python
Progress Report 2	07/03/2025	2			Modified the front_end script to generate SAS image URL instead of public image URL - Now the product image is visible in the Web Portal
Progress Report 2	08/03/2025	3			Tested the performance of 2 MiniLM Models - 'all-MiniLM-L12-v2' & 'all-MiniLM-L6-v2'
Progress Report 2	09/03/2025	4			Followed the course - Natural Language Processing: NLP With Transformers in Python Tested the performance of 'all-mpnet-base-v2' model yielding the best accuracy so far
Progress Report 2	10/03/2025	1			Followed the course - Natural Language Processing: NLP With Transformers in Python
Progress Report 2	11/03/2025	1			Tested the performance of 9 SBERT models using Hugging Face's Transformers library
Progress Report 2	12/03/2025	2			Tested the performance of 9 SBERT models using Hugging Face's Transformers library Followed the course - Natural Language Processing: NLP With Transformers in Python
Progress Report 2	13/03/2025	2			Tested the performance of 9 SBERT models using Hugging Face's Transformers library Followed the course - Natural Language Processing: NLP With Transformers in Python
Progress Report 2	14/03/2025	1			Followed the course - Natural Language Processing: NLP With Transformers in Python
Progress Report 2	15/03/2025	4			Finalized the scripts - model_evaluation_pc & model_evaluation_pd to evaluate the performance of 9 SBERT models using Hugging Face's Transformers library
Progress Report 2	16/03/2025	4			Drafted and finalized the progress report 1

Link to Work Logs Sheet - <https://docs.google.com/spreadsheets/d/1GEJrsX2NxDs6eYnWCq-M8xBMALygyzSMiK9KGYyX6F8/edit?pli=1&gid=0#gid=0>

Summary

1. Demo & Feedback

- Conducted a demo for the manager at PCA and gathered feedback.

2. Research & Learning

- Researched text similarity computation using NLP, specifically **SBERT (Sentence-BERT) models**.
- Studied SBERT (Sentence-BERT) models available on **Hugging Face's Transformers library** for sentence embeddings and text similarity, including:
 - **MiniLM Models:** 'all-MiniLM-L12-v2' & 'all-MiniLM-L6-v2'
 - **MPNet-Based Models:** 'all-mpnet-base-v2'
 - **BERT-Based Models:** 'bert-base-nli-mean-tokens' & 'bert-large-nli-stsb-mean-tokens'
 - **DistilBERT-Based Models:** 'distilbert-base-nli-stsb-mean-tokens'

- **RoBERTa-Based Models:** ‘roberta-base-nli-stsb-mean-tokens’ & ‘roberta-large-nli-stsb-mean-tokens’
- **Multi-Lingual Models:** ‘paraphrase-multilingual-MiniLM-L12-v2’
- Enrolled in and followed the Udemey course "[Natural Language Processing: NLP With Transformers in Python](#)"

[Read more in 'Work Logs - Detailed' section](#)

3. Model Implementation & Evaluation

- Modified the **ml_model** script to consider all textual fields (product_name & product_description).
- Finalized model evaluation scripts - **model_evaluation_pc** & **model_evaluation_pd** for comparing SBERT models.
- Evaluated the performance of 9 SBERT models, identifying ‘**all-mpnet-base-v2**’ as the best model for product name/category (shorter text) and ‘**bert-large-nli-stsb-mean-tokens**’ as the best model for product description (longer text).
- Based on the above decision, finalized the text similarity part of the **ml_model** script.

[Read more in 'Work Logs - Detailed' section](#)

4. Web Portal Enhancement

- Modified the **front_end** script to generate SAS image URLs instead of public image URLs, ensuring product images are visible in the web portal.

5. Reporting & Documentation

- Drafted and finalized **Progress Report 2** summarizing the key developments.

Work Logs – Detailed

Research & Learning

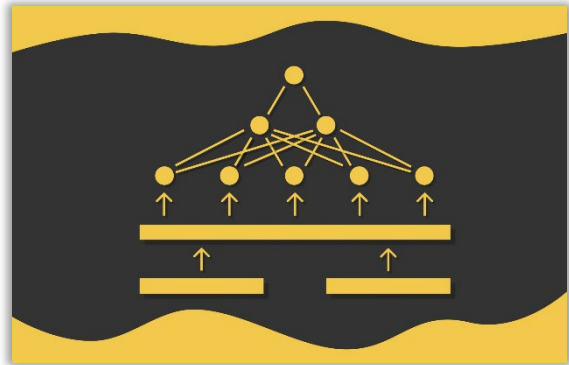
Word and Sentence Embedding Models in NLP

I researched various word and sentence embedding models used in Natural Language Processing (NLP). These models transform words, sentences, or documents into numerical vectors that represent their semantic meaning. They are widely applied in tasks such as semantic search, text classification, similarity analysis, machine translation, and information retrieval.

Model Name	Embedding Type	Best For	Strengths	Weaknesses
Universal Sentence Encoder (USE)	Sentence-level	General sentence embeddings, similarity, retrieval	Fast, pre-trained on diverse data, works well for sentence similarity	Not as customizable, slightly heavier than some models
fastText (Meta)	Word-level	Word-level similarity, fast text classification	Handles out-of-vocabulary words, fast and efficient	Limited to word-level embeddings, does not capture sentence context
GloVe (Stanford)	Word-level	Word similarity, NLP tasks requiring static embeddings	Pre-trained on large corpora, widely used	Static embeddings, lacks context-awareness
Word2Vec (Google)	Word-level	Word similarity, analogies, general NLP tasks	Efficient, simple, works well for many NLP tasks	Does not consider word order or context variations
Doc2Vec (Google)	Document-level	Document classification, topic modeling	Captures document-level semantics	Training can be complex, requires large datasets
ELMo (AllenNLP)	Word-level (contextual)	Contextual word embeddings, word sense disambiguation	Deep contextual understanding, considers word meanings in context	Computationally expensive, requires significant resources
OpenAI Ada Embeddings	Sentence-level	General NLP tasks, embeddings for large-scale applications	High-quality embeddings optimized for large-scale applications	Proprietary, expensive, requires API calls
SBERT (Sentence-BERT)	Sentence-level	Sentence similarity, retrieval, paraphrase detection	State-of-the-art performance in sentence similarity and retrieval	Computationally expensive for large-scale processing

Why SBERT (Sentence-BERT)?

- **SBERT (Sentence-BERT)** is the best choice for **sentence similarity** due to its state-of-the-art performance in semantic similarity tasks. It is specifically designed for efficient sentence embedding and retrieval. Unlike traditional BERT, which processes text without optimized sentence-level representations, SBERT uses **Siamese and triplet networks** to generate meaningful sentence embeddings.
- **Universal Sentence Encoder (USE)** is another strong contender, especially for **general-purpose sentence embeddings**, but SBERT typically outperforms USE in similarity and retrieval tasks.
- **OpenAI Ada Embeddings** provide high-quality embeddings but require API calls and are proprietary, making them **expensive** for large-scale applications.



What is Hugging Face?

Hugging Face is a **leading AI** platform that provides an extensive library of pretrained transformer models, including SBERT, through its Transformers and Sentence-Transformers libraries. It serves as a **hub for easy access to state-of-the-art NLP models**, allowing users to fine-tune and deploy models efficiently.

Why Use Hugging Face for SBERT?

- **Easy Access to SBERT Models** – Hugging Face hosts multiple pretrained SBERT variations (e.g., all-MiniLM-L6-v2, paraphrase-MPNet-base-v2), making it easy to choose the best model for different sentence similarity tasks.
- **Seamless Integration** – The sentence-transformers library, maintained by Hugging Face, allows direct usage of SBERT with minimal coding effort.
- **Fine-Tuning & Customization** – Users can fine-tune SBERT for domain-specific tasks without training from scratch.
- **Optimized for Scalability** – Hugging Face enables cloud-based inference, model sharing, and API-based deployment, making it suitable for production systems.
- **Active Community & Support** – With frequent updates and community contributions, Hugging Face ensures that SBERT models are well-maintained and up to date.

SBERT Models available on SentenceTransformer Library in Hugging Face

Model Name	Speed (Relative)	Accuracy (STS Benchmark*)	Dimensionality	Best Use Case
all-mpnet-base-v2	★ ★	83.1	768	High-accuracy semantic search, NLP applications
roberta-large-nli-stsb-mean-tokens	★	81.5	1024	High-quality text similarity, Computationally expensive
bert-large-nli-stsb-mean-tokens	★	79.2	1024	High-accuracy semantic similarity, Requires more computation
all-MiniLM-L12-v2	★ ★ ★ ★	79	384	Balanced speed & accuracy, Moderate-sized applications
all-MiniLM-L6-v2	★ ★ ★ ★ ★	78.6	384	General-purpose, Fast similarity search, Small-scale applications
roberta-base-nli-stsb-mean-tokens	★ ★ ★	78.5	768	RoBERTa-based semantic similarity, Robust performance
paraphrase-multilingual-MiniLM-L12-v2	★ ★ ★ ★	78	384	Multilingual sentence embeddings, Fast performance
bert-base-nli-mean-tokens	★ ★ ★	77.2	768	Standard semantic similarity tasks, Moderate performance
distilbert-base-nli-stsb-mean-tokens	★ ★ ★ ★	76.7	768	Efficient similarity search, Good balance between speed and accuracy

* The STS (Semantic Textual Similarity) Benchmark is a widely used dataset and evaluation metric for assessing the performance of models in determining the semantic similarity between two text inputs.

Based on the table, the most accurate SBERT models are ***all-mpnet-base-v2***, ***roberta-large-nli-stsb-mean-tokens*** and ***bert-large-nli-stsb-mean-tokens***. However, since our project requires both high accuracy and fast performance, we need to evaluate these models using a real dataset - comprising product categories and descriptions from the product guide - to determine the best model for our needs.

Model Implementation & Evaluation

In text similarity models like SBERT, we use Mean Reciprocal Rank (MRR) and Top-K Accuracy to evaluate how well the model ranks the correct or most relevant matches when retrieving similar texts from a database.

Evaluation Metric – Mean Reciprocal Rank (MRR)

- **Purpose:** MRR measures how well the model ranks the first correct answer in a list of retrieved candidates.
- **Why it matters:** In applications like **product search, FAQ retrieval, and recommendation systems**, we care about whether the correct answer appears at the **top** of the ranked list.

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$$

where $rank_i$ is the position of the first relevant document in the ranked list for the i -th query.

Interpretation:

- A **higher MRR (closer to 1)** means the model ranks correct answers at the top.
- A **lower MRR (closer to 0)** indicates that relevant documents are often found lower in the ranking.

Evaluation Metric – Top-K Accuracy

- **Purpose:** Measures whether at least one relevant document appears in the top K results.
- **Why it matters:** In real-world use cases like **search engines, chatbots, or document retrieval**, users expect to find the correct result in the top few ranked outputs (e.g., top 3 or top 10).

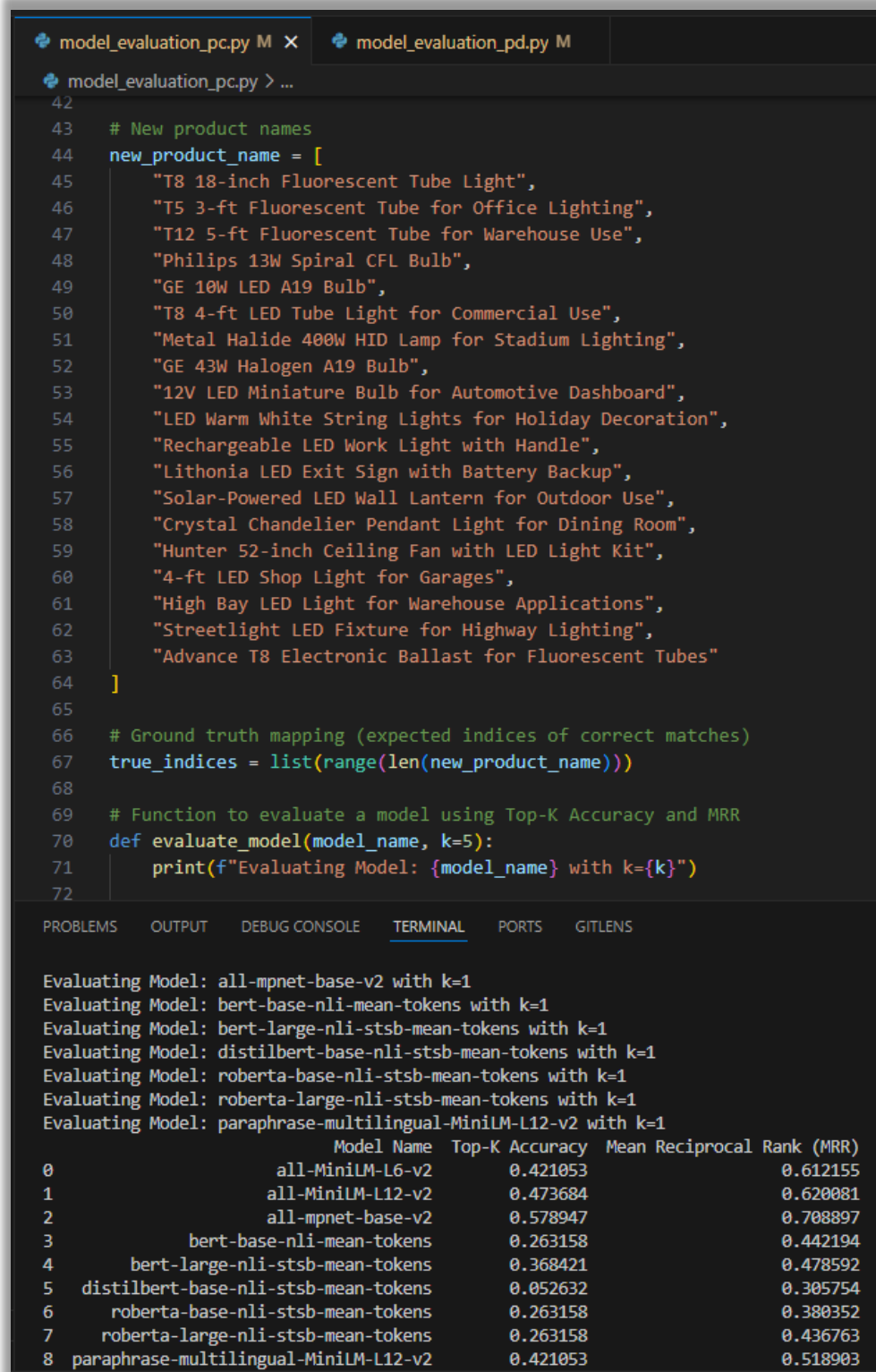
$$\text{Top-K Accuracy} = \frac{\text{Number of queries with at least one correct result in top K}}{\text{Total number of queries}}$$

Common values of K:

- **Top-1 Accuracy:** Did the model get the correct match as the first result?
- **Top-3 Accuracy:** Was the correct answer in the top 3 results?
- **Top-5 Accuracy:** Did it appear in the top 5?

Evaluating 9 SBERT Models using Sample Products

All models were assessed using a sample dataset containing product information. Each new product entry included a Product Name and Product Description, referred to as query sentences. These new products were compared against existing product records, which contained a Product Category and Product Description, referred to as comparison sentences.



```

model_evaluation_pc.py M X  model_evaluation_pd.py M
model_evaluation_pc.py > ...
42
43 # New product names
44 new_product_name = [
45     "T8 18-inch Fluorescent Tube Light",
46     "T5 3-ft Fluorescent Tube for Office Lighting",
47     "T12 5-ft Fluorescent Tube for Warehouse Use",
48     "Philips 13W Spiral CFL Bulb",
49     "GE 10W LED A19 Bulb",
50     "T8 4-ft LED Tube Light for Commercial Use",
51     "Metal Halide 400W HID Lamp for Stadium Lighting",
52     "GE 43W Halogen A19 Bulb",
53     "12V LED Miniature Bulb for Automotive Dashboard",
54     "LED Warm White String Lights for Holiday Decoration",
55     "Rechargeable LED Work Light with Handle",
56     "Lithonia LED Exit Sign with Battery Backup",
57     "Solar-Powered LED Wall Lantern for Outdoor Use",
58     "Crystal Chandelier Pendant Light for Dining Room",
59     "Hunter 52-inch Ceiling Fan with LED Light Kit",
60     "4-ft LED Shop Light for Garages",
61     "High Bay LED Light for Warehouse Applications",
62     "Streetlight LED Fixture for Highway Lighting",
63     "Advance T8 Electronic Ballast for Fluorescent Tubes"
64 ]
65
66 # Ground truth mapping (expected indices of correct matches)
67 true_indices = list(range(len(new_product_name)))
68
69 # Function to evaluate a model using Top-K Accuracy and MRR
70 def evaluate_model(model_name, k=5):
71     print(f"Evaluating Model: {model_name} with k={k}")
72
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

Evaluating Model: all-mpnet-base-v2 with k=1
Evaluating Model: bert-base-nli-mean-tokens with k=1
Evaluating Model: bert-large-nli-stsb-mean-tokens with k=1
Evaluating Model: distilbert-base-nli-stsb-mean-tokens with k=1
Evaluating Model: roberta-base-nli-stsb-mean-tokens with k=1
Evaluating Model: roberta-large-nli-stsb-mean-tokens with k=1
Evaluating Model: paraphrase-multilingual-MiniLM-L12-v2 with k=1

```

	Model Name	Top-K Accuracy	Mean Reciprocal Rank (MRR)
0	all-MiniLM-L6-v2	0.421053	0.612155
1	all-MiniLM-L12-v2	0.473684	0.620081
2	all-mpnet-base-v2	0.578947	0.708897
3	bert-base-nli-mean-tokens	0.263158	0.442194
4	bert-large-nli-stsb-mean-tokens	0.368421	0.478592
5	distilbert-base-nli-stsb-mean-tokens	0.052632	0.305754
6	roberta-base-nli-stsb-mean-tokens	0.263158	0.380352
7	roberta-large-nli-stsb-mean-tokens	0.263158	0.436763
8	paraphrase-multilingual-MiniLM-L12-v2	0.421053	0.518903

The matching process was based on the following criteria:

- **Criteria 1:** The Product Name of the new product was matched against the Product Category of the existing product in the product guide (**shorter text**).
- **Criteria 2:** The Product Description of the new product was matched against the Product Description of the existing product in the product guide (**longer text**).

All 9 models were evaluated based on these two matching criteria. The results are presented below.

In our case,

- MRR helps check if the correct product match appears as the **first result**.
- Top-K Accuracy helps determine if the right product appears **within the top K results**, ensuring web portal admin gets relevant suggestions.

Since our primary goal is to retrieve the correct product match as the top result, **Mean Reciprocal Rank (MRR) and Top-1 Accuracy are the most suitable metrics** for evaluating our models.

Criteria 1 – Shorter Text (Product Name)

Model Name	Mean Reciprocal Rank (MRR)	Top-1 Accuracy	Top-3 Accuracy	Top-5 Accuracy
all-mpnet-base-v2	0.7089	0.5789	0.7895	0.8421
all-MiniLM-L12-v2	0.6201	0.4737	0.7368	0.7895
all-MiniLM-L6-v2	0.6122	0.4211	0.7895	0.8421
paraphrase-multilingual-MiniLM-L12-v2	0.5189	0.4211	0.4737	0.6842
bert-large-nli-stsb-mean-tokens	0.4786	0.3684	0.4737	0.4737
bert-base-nli-mean-tokens	0.4422	0.2632	0.5263	0.6842
roberta-large-nli-stsb-mean-tokens	0.4368	0.2632	0.5263	0.6842
roberta-base-nli-stsb-mean-tokens	0.3804	0.2632	0.3158	0.5263
distilbert-base-nli-stsb-mean-tokens	0.3058	0.0526	0.4737	0.4737

Based on the provided table, the best model for (shorter) text similarity is “all-mpnet-base-v2” because it has the highest values across all key evaluation metrics:

- **Mean Reciprocal Rank (MRR): 0.7089 (highest among all models)**

- **Top-1 Accuracy: 0.5789 (highest)**
- **Top-3 Accuracy: 0.7895 (highest)**
- **Top-5 Accuracy: 0.8421 (highest)**

This indicates that **all-mpnet-base-v2** is the most accurate model **for shorter text similarity tasks**. MPNet-based models outperform others in short-form text similarity tasks due to their better contextual embeddings.

Criteria 2 – Longer Text (Product Description)

Model Name	Mean Reciprocal Rank (MRR)	Top-1 Accuracy	Top-3 Accuracy	Top-5 Accuracy
bert-large-nli-stsb-mean-tokens	0.6899	0.6182	0.7273	0.7636
all-MiniLM-L6-v2	0.6503	0.5455	0.7273	0.7818
all-MiniLM-L12-v2	0.6409	0.5455	0.7091	0.7636
all-mpnet-base-v2	0.6294	0.5273	0.6727	0.7818
paraphrase-multilingual-MiniLM-L12-v2	0.6236	0.5455	0.6727	0.7091
roberta-large-nli-stsb-mean-tokens	0.6107	0.5091	0.6909	0.6909
roberta-base-nli-stsb-mean-tokens	0.6106	0.5273	0.6364	0.7091
bert-base-nli-mean-tokens	0.5884	0.4727	0.6545	0.7091
distilbert-base-nli-stsb-mean-tokens	0.5713	0.4545	0.6545	0.7091

Based on the table, “bert-large-nli-stsb-mean-tokens” is the best model for ensuring correct product matches appear as the first result because:

1. Highest Mean Reciprocal Rank (MRR):

- MRR measures ranking quality, particularly how early the correct match appears.
- The highest MRR (0.6899) suggests this model ranks correct matches higher than others.

2. Highest Top-1 Accuracy:

- This indicates how often the correct match appears as the first result.
- At **0.6182**, it outperforms all other models in getting the first result correct.

For longer text similarity tasks, BERT-based models (especially the larger versions) perform well due to their ability to capture contextual information effectively. Since [bert-large-nli-stsb-mean-tokens](#) leads in both key metrics, it is the best choice **for longer text similarity tasks**.

Final Model Selection for Text Similarity

The above results further confirm that ***all-mpnet-base-v2*** and ***bert-large-nli-stsb-mean-tokens*** are the top-performing models for Product Name/Category and Product Description respectively among the evaluated SBERT models, as also validated by the [STS Benchmark](#).

Once a product inquiry is submitted, the ML system will process it using two models: *all-mpnet-base-v2* for analyzing the Product Name/Category and *bert-large-nli-stsb-mean-tokens* for evaluating the Product Description.

- If both models identify the same product, its category will be returned directly to the backend.
- If the models return different products, the system will select the one with the highest similarity score and return its category to the backend.

The retrieved product category will then be stored in Azure Tables and made available for the administrator's review and approval within the web portal.

Repository Check-in Details

The scripts I have checked into the GitHub repository since the Midterm Report are as follows:

- [front_end script](#) – Updated to generate SAS image URLs instead of public image URLs.
- [model_evaluation_pc script](#) – Created to test performance of 9 SBERT models with Hugging Face Sentence Transformers Library using Product Category.
- [model_evaluation_pd script](#) – Created to test performance of 9 SBERT models with Hugging Face Sentence Transformers Library using Product Description.
- [ml_model script](#) – Updated to accommodate the best SBERT models with Hugging Face Sentence Transformers Library.

Regular check-ins were maintained to ensure all progress was tracked in the repository.

References

- Pretrained Models - Sentence Transformers documentation:
https://www.sbert.net/docs/sentence_transformer/pretrained_models.html
- Large Language Models: SBERT – Sentence-BERT: <https://towardsdatascience.com/sbert-deb3d4aef8a4/>
- Hugging Face Models: <https://huggingface.co/models>
- What is Sentence Similarity: <https://huggingface.co/tasks/sentence-similarity>
- Hugging Face Sentence Similarity Models:
https://huggingface.co/models?pipeline_tag=sentence-similarity&sort=trending
- Natural Language Processing: NLP With Transformers in Python Udemy Course:
<https://www.udemy.com/course/nlp-with-transformers/>
- Vector Similarity Search and FAISS YouTube Course:
<https://www.youtube.com/playlist?list=PLIUOU7oqGTLhIWpTz4NnuT3FekouIVlqc>
- ChatGPT: OpenAI